eman ta zabal zazu

Universidad del País Vasco
Euskal Herriko Unibertsitatea

Sistemas de Gestión de Seguridad

Grado en Ingeniería Informática de Gestión y sistemas de información

Sistema Web: Auditoría

2025-2026

Fecha:

Bilbao, a 30 de octubre de 2025

Grupo:
Podcast
Autores:
Fernández Pajares, María
Fernández Molano, Iker
Galván Carrillo, Nahia
Molinero Medel, Lucía
Rivera Urretxi, Aitzol
Tapias Monasterio, Paula

ÍNDICE

1. ROTURA DE CONTROL DE ACCESO..... 2
1.1. CROSS-SITE REQUEST FORGERY (CSRF).....2
1.2. ACCESO Y PERMISO DE MODIFICACIÓN DE DATOS DE OTROS USUARIOS2
2. FALLOS CRIPTOGRÁFICOS..... 3
2.1. CONTRASEÑA SIN PBKDFs Y TEXTO PLANO.....3
2.2. CONEXIONES INSEGURAS CON LA BASE DE DATOS.....3
3. INYECCIÓN.....4
3.1. INYECCIÓN SQL.....4
4. DISEÑO INSEGURO.....5
4.1 FALTA DE MEDIDAS DE PROTECCIÓN CONTRA LOS BOTS.....5
4.2 AUSENCIA DE LÍMITES Y CONTROLES SOBRE EL USUARIO..... 5
4.3 VALIDACIONES DE DATOS SÓLO EN EL LADO DEL CLIENTE..... 5
4.4 MEDIDAS COMO TOKENS CSRF, RATE-LIMITING O VERIFICACIÓN POR ROLES.....5
4.5 ESTRATEGIA DE GESTIÓN DE ERRORES.....5
5. CONFIGURACIÓN DE SEGURIDAD INSUFICIENTE.....6
5.1. COOKIES DE SESIÓN SIN FLAGS SEGUROS Y SIN REGENERACIÓN DE ID. 6
5.2. SERVIDOR SOBRE HTTP (SIN TLS/HTTPS) Y SIN HTTPS..... 6
5.3. CABECERAS DE SEGURIDAD HTTP AUSENTES (CSP, X-Frame-Options, X-Content-Type-Options, REFERRER-POLICY).....7
6. COMPONENTES VULNERABLES Y OBSOLETOS.....8
6.1. DESCONOCIMIENTO DE VERSIONES EXACTAS.....8
6.2. USO DE SOFTWARE OBSOLETO O SIN SOPORTE.....8
6.3. DEPENDENCIAS EXTERNAS NO CONTROLADAS..... 8
7. FALLOS DE IDENTIFICACIÓN Y AUTENTICACIÓN..... 9
7.1. AUSENCIA DE AUTENTICACIÓN..... 9
7.2. FALTA DE LIMITACIÓN DE INTENTOS Y MENSAJES..... 9
8. FALLOS EN LA INTEGRIDAD DE DATOS Y SOFTWARE.....10
8.1. INCORPORACIÓN DE RECURSOS EXTERNOS SIN VERIFICAR SU INTEGRIDAD.....10
8.2. FALTA DE VERIFICACIÓN DE INTEGRIDAD DEL SOFTWARE LOCAL.....10
9. FALLOS EN LA MONITORIZACIÓN DE LA SEGURIDAD..... 11
9.1. ALMACENAMIENTO INADECUADO DE EVENTOS AUDITABLES..... 11
9.2. ADVERTENCIAS Y ERRORES REGISTRADOS DE MANERA INCORRECTA.. 11
9.3. FALTA DE MONITORIZACIÓN EN TIEMPO REAL..... 11
9.4. FALTA DE PROCESOS AUTOMÁTICOS DE ALERTA Y RESPUESTA.....11
10. BIBLIOGRAFÍA.....12

1. ROTURA DE CONTROL DE ACCESO

1.1. CROSS-SITE REQUEST FORGERY (CSRF)

El ataque CSRF o en español Falsificación de Petición en Sitios Cruzados aprovecha la sesión activa de un sitio web para desde otro, mientras la sesión siga activa, realizar cambios no autorizados en el primero.

En nuestro sitio web, iniciamos una sesión desde login, pero no se genera un token CSRF, necesario para evitar este tipo de vulnerabilidades.

Al generar un token CSRF, la sesión guarda este valor y, a su vez, cada formulario debe contener también el token, pero de manera oculta. Cuando se reciba una petición POST, se verificará si ambos tokens coinciden; si coinciden, la solicitud se procesará.

Esta verificación impide que un sitio web externo, aún teniendo la cookie de sesión, pueda realizar acciones no autorizadas, ya que sólo el dominio asociado a la cookie puede verla y modificarla.

1.2. ACCESO Y PERMISO DE MODIFICACIÓN DE DATOS DE OTROS USUARIOS

Aunque en nuestro sistema web haya una sesión, es posible acceder a los datos e incluso modificarlos de otro usuario. Por ejemplo, en show_user, la URL es localhost:81/show_user?user={usuario} y el contenido mostrado se basa según el valor de {usuario}. Es decir, si se conoce el identificador, el usuario, de otra persona, se puede acceder a sus datos y una vez dentro, también podrían modificarse sus datos.

2. FALLOS CRIPTOGRÁFICOS

2.1. CONTRASEÑA SIN *PBKDFs* Y TEXTO PLANO

En la página de registro de nuestro sitio web, identificamos que la contraseña es almacenada en texto plano. Es decir, se guarda literalmente, en la base de datos, como el usuario la ha escrito.

Esto representa un gran riesgo, pues cualquiera con acceso a esa información (Data breach) sería capaz de acceder en cualquier cuenta (Account takeover).

Con el uso de *password-based key derivation functions (PBKDFs)*, es decir, contraseñas generadas por funciones, se solucionaría este problema. Una práctica recurrente es generar un hash de la contraseña, el cual es el que se almacena. Hay diferentes algoritmos de generación de hash, en php, por ejemplo, bcrypt o Argon2. Además, añaden un salt aleatorio para diferenciar las contraseñas en caso de que dos usuarios tuviesen la misma.

2.2. CONEXIONES INSEGURAS CON LA BASE DE DATOS

En el sitio web, se establecen conexiones inseguras con la base de datos, lo que hace vulnerable al sistema a ataques Man in the Middle (MITM), donde el atacante intercepta los datos mientras se transmiten.

Las conexiones establecidas con la base de datos no utilizan ningún protocolo de cifrado. Es necesario configurar la conexión como TLS/SSL, para así conseguir una conexión segura donde los datos viajen cifrados.

3. INYECCIÓN

3.1. INYECCIÓN SQL

El ataque por inyección SQL se basa en la vulnerabilidad, que en caso de no tenerla presente, es generada por las diferentes operaciones en la base de datos.

La vulnerabilidad surge cuando se insertan parámetros en una instrucción SQL, los cuales no han sido previamente comprobados o filtrados. Al no existir comprobación previa, es posible insertar código o nuevas instrucciones en el lugar del parámetro. A esta porción, se le conoce como "invasora". Al ejecutar la instrucción SQL, la base de datos puede quedar vulnerable.

En nuestra web, inconscientemente hemos evitado este problema, por ejemplo, en la pestaña add_item realizamos:

```
$stmt = mysqli_prepare($conexion, "INSERT INTO item(nombre, año, combustible, caballos, precio) VALUES (?,?,?,?,?)");

mysqli_stmt_bind_param($stmt, "sisid", $nombre, $anio, $combustible, $caballos, $precio);

mysql_stmt_execute($stmt);
```

En la primera línea, preparamos la consulta, que se usa para evitar que se inserten comandos dentro de la consulta, ya que, inserta de manera literal los parámetros de la segunda línea. Habría riesgo de inyección SQL si la instrucción SQL se diese de la siguiente manera:

```
$stmt = "INSERT INTO item(nombre, año, combustible, caballos, precio) VALUES ($nombre, $anio, $combustible, $caballos, $precio)";

mysqli_query($conexion, $stmt);
```

4. DISEÑO INSEGURO

4.1 FALTA DE MEDIDAS DE PROTECCIÓN CONTRA LOS BOTS

El Sistema Web no tiene planificado defensas contra los accesos masivos, como pueden ser los bots. Cualquiera que programe un bot puede hacer muchos intentos con mil combinaciones de usuario y contraseña, podrían saturar el servidor incluso hasta consiguiendo acceder a la web. No tenemos controles captcha, ni rate limiting, ni bloqueo temporal de cuenta, o registros y alertas ante muchas peticiones desde la misma IP.

4.2 AUSENCIA DE LÍMITES Y CONTROLES SOBRE EL USUARIO

El diseño no define límites de tamaño o tipo de datos para el nombre de usuario por ejemplo. Esto supone un fallo ya que no no hemos realizado restricciones que evitan desbordamientos.

4.3 VALIDACIONES DE DATOS SÓLO EN EL LADO DEL CLIENTE

Nuestro sistema comprueba a través del JavaScript si el DNI, el correo, el teléfono tienen el formato adecuado, sin embargo, si alguien desactiva JavaScript estos datos no se comprueban si se ha añadido bien. Es decir, un atacante podría enviar directamente al servidor datos incorrectos o manipulados.

4.4 MEDIDAS COMO TOKENS CSRF, RATE-LIMITING O VERIFICACIÓN POR ROLES

La falta de protección frente a ataques CSRF, como ya hemos mencionado, refleja un error en el diseño. Es decir, nuestro diseño asume que cualquier solicitud enviada es segura, confiando en el navegador, en la sesión y las cookies. No se ha considerado que un atacante pueda aprovechar una sesión activa para ejecutar acciones.

4.5 ESTRATEGIA DE GESTIÓN DE ERRORES

Cuando nuestra base de datos da error, o salta un error interno a la página, no aparece un mensaje de error, sino que aparece directamente el mensaje de error que proporciona el servidor y esto puede revelar información técnica sobre nuestro sistema web como puede ser el nombre de la base de datos. Esto es un gran problema a nivel de diseño.

Esto hace que sea más fácil para los atacantes averiguar cómo está hecho el sistema o encontrar puntos débiles. Lo mejor es desactivar que se muestren los errores cuando ya está todo funcionando en la vida real y guardar esos errores en un archivo seguro. Así, se evita que se filtre información importante del servidor o de la base de datos.

5. CONFIGURACIÓN DE SEGURIDAD INSUFICIENTE

5.1. COOKIES DE SESIÓN SIN FLAGS SEGUROS Y SIN REGENERACIÓN DE ID

En las páginas donde manejamos la sesión del usuario, usamos `session_start()` para que todo funcione bien. Sin embargo, no hemos configurado las cookies de sesión con las opciones de seguridad (secure, httponly, samesite), ni hemos cambiado el ID de sesión después de que alguien se loguea.

Esto quiere decir que la cookie de sesión podría viajar sin protección por HTTP, alguien podría acceder a ella con JavaScript (si httponly no está activado), o un hacker podría usarla de forma incorrecta.

Si la cookie no está marcada como secure, se puede enviar por conexiones sin cifrar y alguien podría interceptarla. Si no usamos httponly, un ataque XSS podría robar la cookie. Y si no generamos un nuevo ID de sesión después de que alguien entra a su cuenta, un atacante podría meter un ID falso antes de que se autentiquen.

La solución para resolver este problema es sencilla: definimos parámetros de cookie seguros en un archivo común, y generamos una nueva sesión cuando el usuario se loguee correctamente.

5.2. SERVIDOR SOBRE HTTP (SIN TLS/HTTPS) Y SIN HTTPS

Para acceder a la web, utilizamos la url localhost:81/…, y al usar HTTP, envía formularios, claves y cookies sin cifrar.

Esto abre la puerta a ataques Man-in-the-Middle (MITM), donde alguien se mete entre el usuario y el servidor para ver o cambiar los datos que viajan.

Sin TLS, cualquiera puede robar las credenciales, tokens y cookies del usuario, lo que puede llevar a que le roben sus cuentas e información importante.

La solución sería configurar TLS en el servidor y hacer que todo el tráfico HTTP se redirija a HTTPS.

5.3. CABECERAS DE SEGURIDAD HTTP AUSENTES (CSP, X-Frame-Options, X-Content-Type-Options, REFERRER-POLICY)

En nuestra web, no se ha detectado un start.php o configuración HTTP central que añada cabeceras de seguridad. Sin estas cabeceras, la web está expuesta al 'clickjacking', 'MIME sniffing', se pueden filtrar los 'referers' y es más fácil que haya ataques XSS.

Sin un CSP y otras cabeceras, los atacantes tienen más formas de meter scripts dañinos, colar la página en iframes de otros o aprovecharse de fallos en el contenido.

La solución sería añadir cabeceras desde el servidor o al inicio de la web.

6. COMPONENTES VULNERABLES Y OBSOLETOS

6.1. DESCONOCIMIENTO DE VERSIONES EXACTAS

En el proyecto no existe un control claro sobre las versiones exactas de los componentes utilizados.

En el archivo docker-compose.yml se observa el uso de imágenes sin versión fija, como phpmyadmin/phpmyadmin:latest, y una referencia genérica a image: web, cuyo origen y versión no están documentados.

Esto implica que, cada vez que se reconstruyen los contenedores, podrían descargarse versiones distintas de las imágenes, incluyendo potencialmente versiones vulnerables.

La ausencia de un inventario de versiones dificulta la trazabilidad y la detección de vulnerabilidades específicas asociadas a cada dependencia.

6.2. USO DE SOFTWARE OBSOLETO O SIN SOPORTE

El entorno de ejecución emplea versiones que ya no tienen soporte oficial ni actualizaciones de seguridad.

El contenedor del servicio web emplea php:7.2.2-apache, una versión de PHP cuyo soporte terminó hace varios años y que tiene numerosas vulnerabilidades conocidas sin soluciones disponibles.

De igual manera, la base de datos funciona sobre mariadb:10.8.2, versión que también es anterior a las que la comunidad actualmente soporta.

Esto convierte al sistema en un entorno potencialmente inseguro, ya que los atacantes podrían explotar vulnerabilidades identificadas y no solucionadas en esas versiones.

6.3. DEPENDENCIAS EXTERNAS NO CONTROLADAS

El proyecto no mantiene un inventario formal de las dependencias utilizadas ni mecanismos para verificar su seguridad o actualización.

No existe un archivo composer.json ni ninguna herramienta de gestión de dependencias que registre qué bibliotecas o extensiones están instaladas en el entorno PHP.

Además, el código incluye recursos externos cargados desde redes públicas, como la hoja de estilos de Font Awesome a través de un CDN (cdnjs.cloudflare.com), sin control de versión ni integridad.

Esto podría permitir que, si el proveedor externo sufre una brecha o actualiza el recurso con código malicioso, la aplicación quede comprometida sin intervención directa del desarrollador.

7. FALLOS DE IDENTIFICACIÓN Y AUTENTICACIÓN

7.1. AUSENCIA DE AUTENTICACIÓN

La web solo utiliza contraseñas como método de autenticación. Si un atacante obtiene la contraseña de un usuario, puede acceder sin restricciones.

Hacer uso de un único factor de autenticación aumenta la posibilidad de accesos no autorizados.

La solución sería implementar más de un método de autenticación, como por ejemplo, el enviar un código al correo electrónico o dispositivo móvil del usuario.

7.2. FALTA DE LIMITACIÓN DE INTENTOS Y MENSAJES

No existe un sistema que limite los intentos fallidos de inicio de sesión. Esto permite que los atacantes puedan intentar contraseñas sin restricción de intentos.

La solución sería implementar límites de intentos de inicio de sesión, y mostrar mensajes como “Usuario o contraseña incorrectos”.

8. FALLOS EN LA INTEGRIDAD DE DATOS Y SOFTWARE

8.1. INCORPORACIÓN DE RECURSOS EXTERNOS SIN VERIFICAR SU INTEGRIDAD

Se incorporan en diversos archivos web librerías externas a través de CDNs, como CSS y JavaScript o FontAwesome, sin implementar procedimientos para verificar la integridad.

Esto quiere decir que la página web tiene plena confianza en el contenido que llega de servidores externos. Si uno de estos recursos es alterado o comprometido, el navegador ejecutará código dañino en nuestra aplicación. Se considera que esta clase de error es un problema de integridad del software (CWE-494, CWE-829).

Las consecuencias pueden ser serias: un atacante tiene la posibilidad de cambiar el aspecto del sitio, poner en marcha códigos para sustraer información de los usuarios o alterar la actividad del sistema sin acceder directamente al servidor.

La solución para solucionar este problema será incluir los atributos `integrity` y `crossorigin` en las etiquetas `<link>` y `<script>`, o bien descargar y alojar los recursos a nivel local en el servidor.

8.2. FALTA DE VERIFICACIÓN DE INTEGRIDAD DEL SOFTWARE LOCAL

La web utiliza scripts locales (`js/items.js`, entre otros) sin controles de integridad ni validaciones en el servidor.

Si el servidor se ve comprometido o alguien modifica estos archivos, podrían ejecutarse acciones no autorizadas o manipular los datos que el usuario envía y recibe.

La solución será mantener un control de versiones con firmas o verificaciones de integridad. Restringir los permisos de escritura en el servidor para evitar modificaciones. Aplicar auditorías periódicas o generar checksums de los archivos antes del despliegue.

9. FALLOS EN LA MONITORIZACIÓN DE LA SEGURIDAD

9.1. ALMACENAMIENTO INADECUADO DE EVENTOS AUDITABLES

En la web no se registran correctamente eventos críticos como logins, logins fallidos o transacciones de alto valor. Esto impide auditar incidentes de forma efectiva y dificulta la detección de posibles ataques.

Para solucionarlo, se debería implementar un sistema centralizado de logs, con cifrado y control de acceso, asegurando su disponibilidad y confidencialidad.

9.2. ADVERTENCIAS Y ERRORES REGISTRADOS DE MANERA INCORRECTA

La identificación de anomalías y posibles ataques se ve obstaculizada porque las advertencias y los errores no se registran correctamente o no se registran en absoluto.

Una manera de evitar esto es configurar los logs para que se guarden en un archivo protegido todos los errores y advertencias, sin revelar información confidencial al usuario.

9.3. FALTA DE MONITORIZACIÓN EN TIEMPO REAL

Los logs de APIs y aplicaciones no se supervisan para detectar comportamientos sospechosos en tiempo real, lo que permite que ataques activos pasen desapercibidos y aumente su impacto.

Se recomienda implementar herramientas de monitorización o SIEM que analicen los logs en tiempo real y envíen alertas ante eventos sospechosos.

9.4. FALTA DE PROCESOS AUTOMÁTICOS DE ALERTA Y RESPUESTA

No existen procesos automáticos que generen alertas o respuestas ante determinados umbrales de actividad, y las pruebas de pen-testing no generan notificaciones. Esto retrasa la detección de incidentes y permite que un atacante explote vulnerabilidades sin ser detectado.

Para corregirlo, se deben establecer alertas automáticas y procesos de respuesta basados en eventos críticos y umbrales de actividad, incluyendo la integración de pruebas de penetración periódicas.

10. BIBLIOGRAFÍA

Mikel Egaña Aranguren — Seguridad Web (2024). Seguridad en sistemas Web. Repositorio de Github. Recuperado el 24 de octubre de 2025, de https://github.com/mikel-egana-aranguren/EHU-SGSSI-01/tree/main/Seguridad_web

Wikipedia — Inyección SQL. (s. f.). *Inyección SQL*. Wikipedia, la enciclopedia libre. Recuperado el 28 de octubre de 2025, de https://es.wikipedia.org/wiki/Inyecci%C3%B3n_SQL

Wikipedia — Cross-site request forgery. (s. f.). *Cross-site request forgery*. Wikipedia, the free encyclopedia. Recuperado el 28 de octubre de 2025, de https://en.wikipedia.org/wiki/Cross-site_request_forgery

Stack Overflow — How to use mysqli connection with SSL? (2015). *How to use mysqli connection with SSL?* Stack Overflow. Recuperado el 28 de octubre de 2025, de https://stackoverflow.com/questions/29084442/how-to-use-mysqli-connection-with-ssl

Raúl, H. (26 de febrero de 2017). *¿En qué consisten los ataques Parameter Tampering y cómo pueden evitarse?* HenryRaul (blog). Recuperado el 28 de octubre de 2025, de https://henryraul.wordpress.com/2017/02/26/en-que-consisten-los-ataques-parameter-tampering-y-como-pueden-evitarse/

Wikipedia — HTTP Strict Transport Security. (s. f.). HTTP Strict Transport Security. Wikipedia, the free encyclopedia. Recuperado el 29 de octubre de 2025, de https://en.wikipedia.org/wiki/HTTP_Strict_Transport_Security

PHP Documentation — session_set_cookie_params. (s. f.). session_set_cookie_params — Set the session cookie parameters. PHP.net. Recuperado el 29 de octubre de 2025, de https://www.php.net/manual/en/function.session-set-cookie-params.php

Wikipedia — HTTP Strict Transport Security. (s. f.). HTTP Strict Transport Security. Wikipedia, the free encyclopedia. Recuperado el 29 de octubre de 2025, de https://en.wikipedia.org/wiki/HTTP_Strict_Transport_Security

Wikipedia — Content Security Policy. (s. f.). Content Security Policy. Wikipedia, the free encyclopedia. Recuperado el 29 de octubre de 2025, de https://en.wikipedia.org/wiki/Content_Security_Policy

Stack Overflow — How to use mysqli connection with SSL? (2015). How to use mysqli connection with SSL? Stack Overflow. Recuperado el 29 de octubre de 2025, de https://stackoverflow.com/questions/29084442/how-to-use-mysqli-connection-with-ssl

WebDev: Cookies. https://web.dev/articles/samesite-cookies-explained?hl=es

OWASP Foundation — OWASP Top 10.(2021). Las Diez Vulnerabilidades más Críticas en Aplicaciones Web**.** OWAPS Foundation. Recuperado el 30 de octubre de 2025, de https://owasp.org/Top10/es/